

AI TRAINING – DAY 4 ASSIGNMENT

Python Conditional Logic: Decide → Compare → Combine → Nest

Practical + MCQ + Logic Tracing + Debugging + 3 Daily Programs

Level	Beginner → Medium
Suggested Time	3–4 hours
Core Topics	if, elif, else, comparison operators, logical operators, nested conditions
Daily Deliverable	3 completed conditional programs
Main Focus	Learn to design and trace decision logic

Learning Goals

- Understand how Python makes decisions using if, elif and else.
- Use comparison operators correctly: ==, !=, >, <, >= and <=.
- Combine conditions using and, or and not.
- Understand when and why nested conditions are useful.
- Trace a program and predict which branch will execute.
- Identify common conditional-logic errors.
- Build three practical programs independently.
- Explain the logic in your own words.

Assignment Rules

- Write the logic yourself before asking AI for help.
- Before running, predict the output for at least 2 inputs.
- Use meaningful variable names.
- Test boundary values such as 0, 18, 35, 50, 60, 75 and 90 where relevant.
- Avoid giant, unreadable conditions.
- Document Input → Conditions → Output.
- Record one bug you faced and explain how you fixed it.

Part A – Understand the Decision Structure

Structure	Meaning	Your Task
if	Runs a block when a condition is True.	If is conditional statement it runs when the given condition is correct.
if / else	Chooses between two paths.	If/else is also a conditional statement. If the given statement is correct correct then if block will executes otherwise else block will executes.
if / elif / else	Chooses among multiple possible paths.	If/elif/else is three path .at here taken condition will checks two times once in if block.if the conditionin if block is wrong then it checks the elif block otherwise it will executes the else block.
Nested if	A decision inside another decision.	Nested if is nothing but condition inside condition.there more than one if block.

Part B – Comparison Operators Lab

Predict True or False before running:

- `10 > 5` # True
- `10 < 5` #False
- `10 == 10` #True
- `10 != 10` #False
- `25 >= 25` #True
- `24 <= 20` #False
- `'Python' == 'python'` #False
- `'AI' != 'ML'` #False
- `100 == '100'` #False

Explain especially why `'Python' == 'python'` and `100 == '100'` produce their results.

Ans: Because in 'Python' P is uppercase but in 'python' p is lowercase. 100 is integer and '100' is a string both are not equal.

Part C – Logical Operators Lab

Predict the result:

- `True and True` #True
- `True and False` #False
- `False and True` #False
- `False and False` #False
- `True or False` #true
- `False or False` #False
- `not True` #False
- `not False` #True

Then create your own real-life and Python examples for AND, OR and NOT.

```
coding_class = False
work = True
had_time=True
play_games = not had_time and (coding_class or work)
print(play_games)
```

Part D – Decision Tracing

Do not run these first. Predict the output, then test your answer.

```
1)
age = 20
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Ans:

Input:age=20

Condition checked: its checks the if condition .

Branch executed:if condition is true so it executes the if block.

Output:Adult

```
2)
marks = 82
if marks >= 90:
    print("A")
elif marks >= 75:
    print("B")
else:
    print("C")
```

Ans:

Input:marks=82

Condition checked: its checks the if condition and elif condition.

Branch executed:elif condition is true so it executes the elif block.

Output:B

```
3)
age = 20
has_id = True
if age >= 18 and has_id:
    print("Allowed")
else:
    print("Not allowed")
```

Output: Allowed

```
4)
amount = 5000
if amount >= 10000:
    print("High")
elif amount >= 5000:
    print("Medium")
else:
    print("Low")
```

Ans:

Input:amount=5000

Condition checked: its checks if condition and elif condition.

Branch executed:elif condition is true so it executes the elif block.

Output: Medium

```

5)
username = "admin"
password = "1234"

if username == "admin":
    if password == "1234":
        print("Login successful")
    else:
        print("Wrong password")
else:
    print("Unknown user")

```

Ans:

Input:username = "admin"

Password="1234"

Condition checked: its checks the if condition and nested if condition.

Branch executed:if condition is true so it again checks nested if also true.it exectes the nested if block.

Output: Login successful

For each snippet, write: Input → Condition checked → Branch executed → Output.

Part E – MCQs: Basic

1. 1. Which keyword starts a condition?

A) if B) when C) check D) condition

2. 2. Which keyword handles another condition after if?

A) else-if **B)** elif C) otherwise D) next

3. 3. Which keyword handles the remaining case?

A) default **B)** else C) final D) otherwise

4. 4. Which operator checks equality?

A) = **B)** == C) === D) equals

5. 5. What does > mean?

A) less than B) equal **C)** greater than D) not equal

6. 6. Which operator means AND?

A) & **B)** and C) && D) plus

7. 7. Which operator means OR?

A) or B) || C) | D) either

8. 8. Which operator reverses a Boolean result?

A) reverse **B)** not C) ! D) opposite

9. 9. In a single if/elif/else chain, how many else blocks are normally used?

A) 0 only **B)** exactly one C) many D) two

10. 10. Can an if statement exist inside another if statement?

A) No **B)** Yes C) Only in loops D) Only in functions

Part F – MCQs: Medium

1. If $10 > 5$, what is printed by `if 10 > 5: print('Yes')`? **A)** Yes B) No C) Error D) True

2. Which is a correct age condition? **A)** `if age >= 18:` B) `if age == 18:` C) `if age = 18:` D) `if age greater 18:`

3. In an if/elif chain, what happens after the first True condition? A) All branches run **B)** The first True branch runs and the chain stops C) else always runs D) Program restarts

4. What is $10 > 5$ and $2 > 8$? A) True **B)** False C) 10 D) Error

5. What is $10 > 5$ or $2 > 8$? **A)** True B) False C) 8 D) Error

6. What is `not(10 > 5)`? A) True **B)** False C) None D) Error

7. Which represents age between 18 and 60 inclusive? **A)** `age >= 18 and age <= 60` B) `age >= 18 or age <= 60` C) `age > 18 and age > 60` D) `age < 18 and age > 60`

8. What is a nested condition? A) A loop inside a string **B)** A condition inside another condition C) Two variables D) A function call

9. Nested conditions are useful when: **A)** The second decision depends on the first B) You want no conditions C) Only for printing D) Only for loops

10. A common grade-boundary error is: **A)** Using incorrect `>=` or `>` conditions B) Using `print` C) Using variables D) Using input

Part G – MCQs: Thinking

1. Why does elif order matter? **A)** Python checks from top to bottom and stops at the first True branch B) It does not matter C) It sorts conditions D) It runs randomly

2. If $A \geq 90$ and $B \geq 75$, which should be checked first? A) ≥ 75 **B)** ≥ 90 C) < 75 D) $== 90$

3. `Adult AND has ID` can be written as: **A)** `age >= 18 and has_id` B) `age >= 18 or has_id` C) `age <= 18 and has_id` D) `not age`

4. 'Not a student' can be represented by: A) `is_student` **B)** `not is_student` C) `is_student == True` D) `student not`

5. A menu option 9 should be: A) Ignored **B)** Handled explicitly or rejected as invalid C) Always option 1 D) Automatically accepted

6. Why test boundary values? **A)** To find mistakes around `>=`, `>`, `<=` and `<` B) To make code longer C) To avoid input D) To remove variables

7. Why is `age >= 18 or age <= 60` usually wrong for a range? **A)** It is true for almost every number B) It always returns False C) or cannot be used D) Python does not support ranges

8. When is nested if more readable? **A)** When the second decision depends on the first B) Always C) Never D) Only with strings
9. If many branches become hard to read, what should you do? A) Make one giant condition **B)** Break logic into smaller understandable steps C) Remove tests D) Use random conditions
10. A grade calculator should test: A) Only 100 **B)** Normal, boundary and invalid cases C) Only one mark D) No invalid input

Part H – Daily Deliverable: 3 Conditional Programs

Program 1 – Grade Calculator

- Ask for student name and marks.
- Validate marks from 0 to 100.
- Use if/elif/else with at least 5 grade categories.
- Display the grade and a meaningful message.
- Test 90, 75, 60, 50 and 0.
- Test invalid values such as -5 and 110.

Ans:

```
name=input("enter your name:")
marks=int(input("enter your marks:"))
if marks>=90 and marks<=100:
    print(f"{name} you got A grade")
elif marks <90 and marks>=85:
    print(f"{name} you got B grade")
elif marks <85 and marks>70:
    print(f"{name} you got C grade")
elif marks <70 and marks>=50:
    print(f"{name} you got D grade")
elif marks <50 and marks >=0:
    print(f"{name} better luck next time !!")
else:
    print("invalid marks")
```

Program 2 – Eligibility Checker

Choose one realistic scenario: scholarship, course admission, job interview, driving eligibility, loan/application, or gym membership.

- Use at least 3 inputs.
- Use at least one and condition.
- Use at least one or condition.
- Create at least 3 possible outcomes.
- Write your eligibility rules clearly before coding.
- Test a qualifying case, a non-qualifying case and a boundary case.

Ans:

```

age = int(input("Enter your age: "))
marks = int(input("Enter your marks: "))
has_id = input("Do you have an ID? (yes/no): ")
has_id=has_id.lower()=='yes'
has_admission_letter = input("Do you have an admission letter? (yes/no): ")
has_admission_letter = has_admission_letter.lower()=='yes'
if age >= 18 and marks >= 60 and (has_id or has_admission_letter):
    print("Eligible")
elif age < 18 or marks < 60:
    print("Not Eligible")
else:
    print("Document Required")

```

Program 3 – Simple Menu

- Create at least 4 menu choices.
- Ask the user for a choice.
- Use if/elif/else.
- Handle invalid choices.
- Use at least one nested if.
- Suggested project: Banking Menu — Check Balance, Deposit, Withdraw, Exit. For withdrawal, use a nested condition to check sufficient balance.
- Test every option and at least one invalid option.

Ans:

```

if choice==1:
    amount=int(input('enter the amount to deposited:'))
    total_amount+=amount
    print(f"{amount} has been deposited succesfully")
    print("1.check balance \n 2.exit")
    choice=int(input("enter your choice(1/2):"))
    if choice ==1:
        print(f"remaining balance is :{total_amount}")
    elif choice==2:
        print("thanks for visiting")
elif choice==2:
    amount=int(input("enter amount to be withdrawn:"))
    total_amount-=amount
    choice=int(input("enter your choice(1/2):"))
    if choice ==1:
        print(f"remaining balance is :{total_amount}")
    elif choice==2:
        print("thanks for visiting")
elif choice==3:
    print(total_amount)
elif choice ==4:
    print("thanks for visiting.")
else:
    print("wrong option selected")

```

Part I – Debugging Lab

For every bug: identify the problem, explain why it happens, fix it, and test it.

```
Bug 1
age = 18
if age = 18:
    print("Adult")
```

Why it happens: at "if age = 18:" "=" is used for assignment .to check we need to use "==".

Fix it:

```
age = 18
if age == 18:
    print("Adult")
```

test it: executed succesfully

Hint: What is the difference between = and ==?

```
Bug 2
marks = 85
if marks >= 50:
    print("Pass")
elif marks >= 80:
    print("Excellent")
```

Why it happens: in condition checking we need write high value first after that lower values.

Fix it:

```
marks = 85
if marks >= 80:
    print("Excellent")
elif marks >= 50:
    print("pass")
```

test it: executed succesfully

Why does Excellent never print? Fix the order of conditions.

```
Bug 3
age = 25
if age >= 18 or age <= 60:
    print("Eligible")
```

Why it happens: at if condition we need to use "and" operator because "or" can take any one value as true .we can notice in above program if we enter input 1 it will executes because we give condition age <= 60: for range we use "and".

Fix it:

```
age = 1
if age >= 18 and age <= 60:
    print("Eligible")
else:
    print("not Eligible")
```

test it: executed succesfully

Explain why this is wrong for an 18–60 range and correct it using and.

```
Bug 4
has_id = True
age = 17

if age >= 18 and has_id:
    print("Allowed")
else:
    print("Not allowed")
```

Ans :

Program will execute successfully there is no error.

Predict the output and explain which part of the condition makes it False.

```
Bug 5
choice = 2

if choice == 1:
    print("Deposit")
elif choice == 2:
    print("Withdraw")
elif choice == 2:
    print("Check Balance")
```

Ans

Why it happens: we used two elif blocks instead of that we can use one else block at the ending . and that not a major bug the main problem is with we checks "choice == 2:" this condition for two time .it does not check second time . instead of that we use else block.
"

Fix it:

```
choice = 0

if choice == 1:
    print("Deposit")
elif choice == 2:
    print("Withdraw")
else:
    print("Check Balance")
```

test it: executed succesfully

Why is the final branch unreachable? Redesign the menu conditions.

```
Bug 6
password = "1234"

if password == "1234":
    if password == "admin":
        print("Success")
    else:
        print("Wrong")
```

Why it happens: because we have assigned password as "1234" but we are checing for "admin" this will print wrong.

Fix it:

```
password = "1234"
username="admin"
if password == "1234":
    if username == "admin":
        print("Success")
    else:
        print("Wrong")
```

test it: executed succesfully

Explain the nested logic. Why can this never print Success? Redesign the login logic sensibly.

Part J – Nested Conditions Challenge

Build a Login + Access Control program.

- Ask for username and password.
- If the username is correct, use a nested condition to check the password.
- If both are correct, ask whether the user is an admin.

- Show different messages for normal users and admins.
- Show separate messages for wrong username and wrong password.
- Test at least 5 combinations.

```
username = input("Enter username: ")
password = input("Enter password: ")
if username == "admin":
    if password == "1234":
        is_admin = input("Are you an admin? (yes/no): ")

        if is_admin == "yes":
            print("Welcome Admin")
        else:
            print("Welcome Normal User")
    else:
        print("Wrong password")
else:
    print("Wrong username")
```

input:1

Enter username: admin
Enter password: 1234
Are you an admin? (yes/no): yes

Output:

Welcome Admin

Input:2

Enter username: admin
Enter password: 1234
Are you an admin? (yes/no): no

Output: Welcome Normal User

Input :3

Enter username: deva
Enter password: 1234

Output: Wrong username

Input:3

Enter username: admin
Enter password: 854
Output:

Wrong password

Part K – Real-World Logic Challenge

Choose one: food-delivery charge calculator, movie-ticket pricing, college-attendance eligibility, ATM withdrawal, electricity-bill category, or mobile-data plan.

- Use at least 4 inputs or decision factors.
- Create at least 5 possible outcomes.
- Use comparison and logical operators.
- Use at least one nested condition.
- Draw a decision tree before coding.
- Test normal, boundary and invalid cases.

Inputs:

attendance
marks
has_id
medical

outputs:

Eligible
Not Eligible
ID Required
Eligible for exam based on medical

Decision tree:

Attendance ≥ 75 ?

|
Yes

|

Marks ≥ 50 ?

/

No

|

Not Eligible

\

Yes

|

Has ID?

/

No

|

ID Required

\

Yes

|

Medical?

/

Yes

|

Eligible for
Exam

\

No

|

Eligible

Source code:

```
attendance = float(input("Enter attendance percentage: "))
marks = float(input("Enter marks: "))
has_id = input("Do you have ID? (yes/no): ").lower()
medical = input("Do you have medical proof? (yes/no): ").lower()
if attendance >= 75:
    if marks >= 50:
        if has_id == "yes":
```

```
        if medical == "yes":
            print("Eligible for exam based on medical")
        else:
            print("Eligible")
    else:
        print("ID Required")
else:
    print("Not Eligible")
else:
    print("Not Eligible")
```

test case 1:

Enter attendance percentage: 59

Enter marks: 58

Do you have ID? (yes/no): no

Do you have medical proof? (yes/no): yes

Not Eligible

Test case 2:

Enter attendance percentage: 89

Enter marks: 98

Do you have ID? (yes/no): no

Do you have medical proof? (yes/no): yes

ID Required

Test case 3:

Enter attendance percentage: 98

Enter marks: 98

Do you have ID? (yes/no): yes

Do you have medical proof? (yes/no): no

Eligible

Test case 4:

Enter attendance percentage: 76

Enter marks: 51

Do you have ID? (yes/no): yes

Do you have medical proof? (yes/no): yes

Eligible for exam based on medical

Part L – Draw a Decision Tree

Choose the Grade Calculator or Eligibility Checker.

- Start → Input → First decision → Branches → Next decisions → Final output.

- Your flowchart must match the code you finally submit.
- After coding, compare your flowchart with the actual program.

Eligibility Checker :

Decision tree:

START

|

v

Input Attendance

Input Marks

Input ID

Input Medical Proof

|

v

Attendance ≥ 75 ?

|

+---- No ----> NOT ELIGIBLE

|

Yes

|

v

Marks ≥ 50 ?

|

+---- No ----> NOT ELIGIBLE

|

Yes

|

v

Has ID?

|

+---- No ----> ID REQUIRED

|

Yes

|

v

Medical Proof?

|

+---- Yes ---> ELIGIBLE FOR EXAM

|

No

|

v

ELIGIBLE

|

V
END

Source code:

```
attendance = float(input("Enter attendance: "))
marks = float(input("Enter marks: "))
student_id = input("Do you have ID? (yes/no): ")
medical_proof = input("Do you have medical proof? (yes/no): ")

if attendance >= 75:
    if marks >= 50:
        if student_id.lower() == "yes":
            if medical_proof.lower() == "yes":
                print("ELIGIBLE FOR EXAM")
            else:
                print("ELIGIBLE")
        else:
            print("ID REQUIRED")
    else:
        print("NOT ELIGIBLE")
else:
    print("NOT ELIGIBLE")
```

Part M – Explain Your Code

11. Why did you use if/elif/else instead of only if statements?

Ans: if only checks only one condition. If there is more than one condition.

12. Where did you use comparison operators?

Ans: I use comparison operators at marks and attendance programs.

13. Where did you use and/or/not?

Ans: to check in between more than two operations.

14. Where did you use a nested condition?

Ans: when second condition is based on first condition.

15. Which boundary value caused the most thinking?

Ans: at marks>=75 at there is 75 is eligible or not.

16. What bug did you encounter?

Ans: I encounterd the bug where we need to check higher values first and then lower values.

```
marks = 85
if marks >= 50:
    print("Pass")
elif marks >= 80:
    print("Excellent")
```

17. How did you improve your first version?

Ans: I had improved my first version because I had done programs on input/output and conditional statements and tested with many inputs and edge cases.

18. Which condition would be hardest for another developer to understand?

Ans: if had_id == "yes" and attendance >= 65:

Part N – Self Check

Skill	Yes / Need Practice	Notes
Basic if	yes	Used to check one condition
if/elif/else	yes	Used to check more than one condition
Using == correctly	yes	Yes I can use ==.
>, <, >=, <=	yes	Used for conditional between two values.
and/or	yes	Between two conditions.
not	yes	Reverse of condition.
Nested conditions	yes	Second condition is based on first condition
Tracing conditional programs	Need practise	Need practise
Invalid input/choices	yes	Can identify the invalid inputs.
Boundary testing	Need practise	Can test the boundary values
Completed 3 required programs	yes	completed
Can explain own logic	yes	I can explain me own logic

Submission Checklist

- Comparison operator experiments completed.
- Logical operator experiments completed.
- Decision tracing completed.
- All 30 MCQs answered.
- Grade Calculator completed.
- Eligibility Checker completed.
- Simple Menu completed.
- Debugging Lab completed.
- Nested Login + Access Control challenge completed.
- Decision tree submitted.
- Reflection and self-check completed.

Suggested Evaluation

Area	Weight
Concepts + MCQs	20%
3 Daily Programs	35%
Debugging	15%
Nested Challenge	15%

DAY 4 PRINCIPLE

*Don't just write conditions. Learn to think in decisions:
What is the input? What are the rules? What happens in every possible case?*